

Introduction to numpy :

It's an advanced tool for manipulating mathematical and logical things like of arrays and can be more of useful than actual python lists and that's the reason they are used

Multi dimensional arrays:

The arrays can be any number based on the number of square brackets at the start and end of the arrays and any number of arrays can be made and by using ndim u can get the dimension and by shape function u get the order the matrix or arrays

Slicing:

U can select a part of the array and can get it either a part of it or a single one too and also u can get the number like (start,end ,step) or (start,end) to get a part or single number from the array if u use negative numbers then u get the reverse numbers of the array from end to start

Arithmetic:

U can bodmas symbols and also floor and modulus to get things done just like normal one in a programming language u can expect of

Broadcasting:

It is i think multiplication of arrays and dot product is the matrix multiplication of them and the rules are either one element of shape is one or same as other array

Aggregate Functions:

They are some statistics like mean,sum,median,mode, and also other statistics things which are useful for calculations

Filtering:

They are just if else conditions wrapped with maths formulae that's all

Random Numbers:

They can be used to generate random numbers either an array full of numbers by specifying the range rng=np.random.default_rng random thing or u can shuffle the arrays or may be generate an all round random number and u can use the round function to round the arrays and their numbers too

```
import numpy as np
```

```
# =====  
# 1. CREATING MULTI-DIMENSIONAL ARRAYS  
# =====  
# 3 stocks, 7 days of closing prices  
# 3 rows (stocks) × 7 columns (days)
```

```

prices = np.array([
    [100, 102, 101, 105, 104, 106, 108], # Stock A (Apple-like)
    [50, 51, 52, 50, 51, 53, 52],      # Stock B (Bank-like)
    [200, 198, 202, 205, 204, 207, 210] # Stock C (Energy-like)
])

print("=" * 50)
print("ORIGINAL DATA")
print("=" * 50)
print(f"Prices array:\n{prices}")
print(f"\nDimensions (ndim): {prices.ndim}")
print(f"Shape (rows, columns): {prices.shape}")
print(f"Total elements (size): {prices.size}")

# =====
# 2. SLICING
# =====

print("\n" + "=" * 50)
print("SLICING")
print("=" * 50)

# Get first 2 stocks, last 3 days
slice1 = prices[:2, -3:] # [:2] = first 2 stocks, [-3:] = last 3 days
print(f"First 2 stocks, last 3 days:\n{slice1}")

# Get every alternate day (step=2) for Stock C (row index 2)
slice2 = prices[2, ::2] # Stock C, day 0,2,4,6
print(f"Stock C on alternate days: {slice2}")

# Reverse the entire array (negative step)
reversed_prices = prices[:, ::-1] # Reverse columns (days)
print(f"Prices in reverse chronological order:\n{reversed_prices}")

# Get single element (Stock B, Day 3)
single = prices[1, 3]
print(f"Stock B on Day 3: ${single}")

# =====
# 3. ARITHMETIC OPERATIONS
# =====

print("\n" + "=" * 50)
print("ARITHMETIC")
print("=" * 50)

```

```

# Calculate returns (percentage change)
# Formula: ((price_today - price_yesterday) / price_yesterday) * 100
returns = ((prices[:, 1:] - prices[:, :-1]) / prices[:, :-1]) * 100
print(f"Daily returns (%):\n{np.round(returns, 2)}")

# Floor division (how many whole shares you can buy with $1000)
cash = 1000
shares_possible = cash // prices
print(f"Shares you can buy with ${cash}:\n{shares_possible}")

# Modulus (remaining cash after buying)
remaining = cash % prices
print(f"Remaining cash after buying:\n{np.round(remaining, 2)}")

# Power (compound growth over 5 days)
compounded = prices ** 2 # Squaring for demonstration
print(f"Squared prices (volatility proxy):\n{compounded}")

# =====
# 4. BROADCASTING & DOT PRODUCT
# =====
print("\n" + "=" * 50)
print("BROADCASTING & DOT PRODUCT")
print("=" * 50)

# Broadcasting: Add $1 transaction fee to all prices
fee = 1
with_fee = prices + fee # Fee broadcasts to every element
print(f"Prices + $1 fee:\n{with_fee}")

# Your portfolio holdings (shares of each stock)
holdings = np.array([10, 20, 5]) # 10 shares of A, 20 of B, 5 of C

# DOT PRODUCT: Portfolio value on each day
# (holdings dot prices) = for each day: (10*price_A + 20*price_B + 5*price_C)
portfolio_value = np.dot(holdings, prices) # or holdings @ prices
print(f"\nPortfolio value each day: ${portfolio_value}")

# Broadcasting: Apply a multiplier to each stock's entire price history
multiplier = np.array([1.1, 0.95, 1.2]) # Stock A up 10%, B down 5%, C up 20%
adjusted_prices = prices * multiplier.reshape(3, 1) # Reshape for broadcasting
print(f"\nAdjusted prices after market shift:\n{np.round(adjusted_prices, 2)}")

# =====

```

5. AGGREGATE FUNCTIONS (Statistics)

```
# =====
```

```
print("\n" + "=" * 50)
print("AGGREGATE FUNCTIONS")
print("=" * 50)
```

```
# Across ALL values
```

```
print(f"Overall mean price: ${np.mean(prices):.2f}")
print(f"Overall median price: ${np.median(prices):.2f}")
print(f"Overall standard deviation: ${np.std(prices):.2f}")
```

```
# Per stock (axis=1 = across days for each stock)
```

```
print(f"\nMean price per stock: {np.mean(prices, axis=1)}")
print(f"Median price per stock: {np.median(prices, axis=1)}")
print(f"Volatility (std dev) per stock: {np.std(prices, axis=1):.2f}")
```

```
# Per day (axis=0 = across stocks for each day)
```

```
print(f"\nAverage market price each day: {np.mean(prices, axis=0):.2f}")
```

```
# Sum & Min/Max
```

```
print(f"Total value of all stocks across all days: ${np.sum(prices):.0f}")
print(f"Highest price ever: ${np.max(prices)}")
print(f"Lowest price ever: ${np.min(prices)}")
print(f"Range per stock: {np.ptp(prices, axis=1)}") # ptp = max - min
```

```
# =====
```

6. FILTERING (If-else conditions with math)

```
# =====
```

```
print("\n" + "=" * 50)
print("FILTERING")
print("=" * 50)
```

```
# Find all days where Stock A > $104
```

```
high_days = prices[0] > 104
print(f"Days Stock A > $104: {high_days}")
print(f"Values: {prices[0][high_days]}")
```

```
# Find all prices between $100 and $200
```

```
mask = (prices >= 100) & (prices <= 200)
filtered_prices = prices[mask]
print(f"\nPrices between $100-$200: {filtered_prices}")
```

```
# Where condition (like ternary operator)
```

```
# Mark if a stock had a good day (>2% up)
```

```

good_day = np.where(returns > 2, "GOOD", "normal")
print(f"\nGood day detection:\n{good_day}")

# Boolean filtering for portfolio value
rich_day = portfolio_value > 3500
print(f"Days portfolio > $3500: {rich_day}")
print(f"Portfolio values on rich days: ${portfolio_value[rich_day]}")

# =====
# 7. RANDOM NUMBERS (Modern way)
# =====
print("\n" + "=" * 50)
print("RANDOM NUMBERS")
print("=" * 50)

# Create a random number generator
rng = np.random.default_rng(seed=42) # seed for reproducibility

# Random array (5 random prices between 10 and 100)
random_prices = rng.uniform(10, 100, size=5)
print(f"Random prices: ${np.round(random_prices, 2)}")

# Random integer array (simulate daily volume in thousands)
volume = rng.integers(100, 1000, size=(3, 7)) # 3 stocks, 7 days
print(f"\nTrading volume (shares in thousands):\n{volume}")

# Shuffle an array (simulate random order of days)
days = np.arange(1, 8) # Days 1-7
rng.shuffle(days)
print(f"\nShuffled day order: {days}")

# Random normal distribution (simulate returns)
normal_returns = rng.normal(loc=0.001, scale=0.02, size=100) # mean 0.1%, std 2%
print(f"Sample random returns: {np.round(normal_returns[:5], 4)}")
print(f"Mean of random returns: {np.mean(normal_returns):.4f}")
print(f"Std of random returns: {np.std(normal_returns):.4f}")

# Round the random numbers
rounded_volume = np.round(volume, -2) # Round to nearest hundred
print(f"\nVolume rounded to nearest hundred:\n{rounded_volume}")

# =====
# 8. BONUS: ALL CONCEPTS COMBINED
# =====

```

```

print("\n" + "=" * 50)
print("BONUS: EVERYTHING TOGETHER")
print("=" * 50)

# Simulate a trading strategy
def trading_simulation():
    # Random 30 days of returns for 5 stocks
    rng = np.random.default_rng(42)
    returns_30d = rng.normal(0.001, 0.02, size=(5, 30))

    # Start with $10,000 equally invested
    initial_prices = np.array([100, 50, 200, 150, 75])
    shares = (10000 / 5) / initial_prices # Equal investment

    # Calculate daily prices from returns
    prices_30d = initial_prices.reshape(5, 1) * (1 + returns_30d).cumprod(axis=1)

    # Filter: Find days where ANY stock dropped >3%
    crash_days = np.any(returns_30d < -0.03, axis=0)
    print(f"Days with >3% crash in any stock: {crash_days.sum()} out of 30")

    # Portfolio value over time (dot product)
    portfolio = shares @ prices_30d

    # Statistics
    print(f"Portfolio final value: ${portfolio[-1]:.2f}")
    print(f"Portfolio mean daily: ${np.mean(portfolio):.2f}")
    print(f"Portfolio volatility: ${np.std(portfolio):.2f}")
    print(f"Best day: ${np.max(portfolio):.2f}")
    print(f"Worst day: ${np.min(portfolio):.2f}")

    # Filter portfolio
    great_days = portfolio > (portfolio[0] * 1.05)
    print(f"Days with 5%+ gain: {great_days.sum()}")

    return portfolio

final_portfolio = trading_simulation()

print("\n" + "=" * 50)
print("✅ You've used EVERY concept from your diary!")
print("=" * 50)
print("Concepts demonstrated:")
print("✓ Multi-dimensional arrays | ✓ ndim & shape | ✓ Slicing (pos/neg/step)")

```

```
print("✓ Arithmetic (+, -, *, /, //, %, **) | ✓ Broadcasting | ✓ Dot product")
print("✓ Aggregate (mean, median, std, sum, min, max, ptp) | ✓ Filtering")
print("✓ Random numbers (default_rng, uniform, integers, normal, shuffle)")
print("✓ Rounding | ✓ Where conditions | ✓ Boolean masks")
```

Axis perimeter – it controls whether calculations happen across rows or columns

```
Arr = np.array([[1,2,3],
               [4,5,6]])
```

```
arr.mean(axis=0) #Average down each column : [2.5,3.5,4.5]
```

```
arr.mean(axis=1) #Average across each row [2,5]
```

In finance terms:

axis=0 = average price across each time for each stock

axis=1 = average price across stocks for each day



Key Takeaways From Your Own Understanding

You wrote in your diary:

- *"Broadcasting is multiplication of arrays"* → Close! It's actually automatic stretching of smaller arrays to match larger ones
- *"Filtering is just if-else conditions wrapped with maths"* → Exactly right, and beautifully said
- *"Random numbers can generate arrays by specifying range"* → Yes, and `default_rng()` is the modern way

Intro to pandas:

This is more data oriented thing which is used by most of all people who are associated with data and data cleaning is done by this and it's useful as file reader and modifier than manually checking for small things too

Series:

They are 1D array or one column of data in a spreadsheet or data frame and is same as just an array in python and we can actually manipulate the data as we intend to and another thing is we can use dictionary to assign the values and the keys too and most useful function "loc" which is used when a dictionary is created and only key value of the thing and also "iloc" it is same as index of array or dictionary whatever it is and gives the thing of that index

DataFrames:

This is what the data looks like and it is multi dimensional array to speak of and with some data and u can manipulate it like selecting ,filtering,aggregation,and lastly the most important work “data cleaning” and u can also do anything with the data and it's either gonna be csv or json or any be other type of file and u import them and do ur work with that and pandas enable that freedom for u

Importing:

The most easy of doing import is getting the data in ur file of project or where the code is and so u can access it is easily if noty that's not hard too but don't forget importing the data or file before doing with it haha

Selection:

The selection can be based on index and one thing i liked is it automatically generates the index of thing u write whether u have index or not doesn't really matter at all so u can select them by using loc function and any other column name at top of that file and also iloc and get their index

Filtering:

Same thing of if else and narrowing down the data but now u r doing for the data and hat too just staging the condition without if else things at all

Aggregation:

Same thing as numpy and and it just shows us one value per column and also if u use to_string() function u can get to see the whole data with no dots between them and be sure to use it for smaller data sets if bigger could cause a big problem of a data overload or something and also use only_numerics=true to get the value if ur data contains strings too and also u can get the values for specifically one type too so u can see them closely and observe them

Datacleaning:

It is a thing to filter ot take out the data which is either irrelevant,not noted or just unnecessary so that u can operate smoothly and u can some functions as str.lower to make up for any data inconsistency u r having in the data and u can replace the data and so on depending on the context and requirements of the dat set and goals u have set

```
import pandas as pd
import numpy as np
```

```
# =====
# 1. CREATING DATAFRAMES & SERIES
# =====
print("=" * 60)
print("1. CREATING DATA STRUCTURES")
```



```

print("=" * 60)

# Create a DataFrame from dictionary (like spreadsheet)
stock_data = {
    'Date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05'],
    'Ticker': ['AAPL', 'AAPL', 'GOOGL', 'GOOGL', 'MSFT'],
    'Price': [150.25, 152.50, 140.00, 142.30, 380.00],
    'Volume': [1000, 1200, 800, 950, 500],
    'Sector': ['Tech', 'Tech', 'Tech', 'Tech', 'Tech']
}

df = pd.DataFrame(stock_data)
print("DataFrame (auto-indexed 0-4):")
print(df)

# Series (one column)
prices = df['Price']
print(f"\nSeries - Prices column:\n{prices}")
print(f"Type: {type(prices)}")

# Dictionary to Series
ticker_map = {'AAPL': 'Apple', 'GOOGL': 'Google', 'MSFT': 'Microsoft'}
ticker_series = pd.Series(ticker_map)
print(f"\nSeries from dictionary:\n{ticker_series}")

# =====
# 2. IMPORTING DATA (CSV/JSON)
# =====
print("\n" + "=" * 60)
print("2. IMPORTING DATA")
print("=" * 60)

# Save to CSV first (simulating external file)
df.to_csv('stocks.csv', index=False)
print("✓ Created stocks.csv")

# Import back (like reading any CSV)
df_imported = pd.read_csv('stocks.csv')
print("✓ Imported from CSV:")
print(df_imported.head(2))

# JSON import/export
df.to_json('stocks.json', orient='records')
df_json = pd.read_json('stocks.json')

```

```

print("✓ Imported from JSON")

# =====
# 3. SELECTION (loc & iloc)
# =====
print("\n" + "=" * 60)
print("3. SELECTION (loc vs iloc)")
print("=" * 60)

# Set custom index (date as index for finance)
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)
print("✓ Set Date as index")

# loc = label-based (uses index label/column name)
print(f"\nloc['2024-01-02'] (row by date):\n{df.loc['2024-01-02']}")
print(f"\nloc[:, 'Price'] (all rows, Price column):\n{df.loc[:, 'Price']}")

# iloc = position-based (0,1,2...)
print(f"\niloc[0] (first row):\n{df.iloc[0]}")
print(f"\niloc[:2, :3] (first 2 rows, first 3 columns):\n{df.iloc[:2, :3]}")

# Single value selection
price_jan2 = df.loc['2024-01-02', 'Price']
print(f"\nSingle value (Jan 2 price): ${price_jan2}")

# =====
# 4. FILTERING (If-else without writing if-else)
# =====
print("\n" + "=" * 60)
print("4. FILTERING (The magic you described)")
print("=" * 60)

# Filter: Price > 150
high_prices = df[df['Price'] > 150]
print("Stocks with Price > $150:")
print(high_prices)

# Filter: Multiple conditions (AND)
high_volume = df[(df['Volume'] > 900) & (df['Price'] > 140)]
print("\nStocks with Volume > 900 AND Price > $140:")
print(high_volume)

# Filter: OR condition

```

```
tech_or_high = df[(df['Sector'] == 'Tech') | (df['Price'] > 300)]
print("\nTech sector OR Price > $300:")
print(tech_or_high)
```

```
# Filter using .isin()
tickers = ['AAPL', 'MSFT']
selected = df[df['Ticker'].isin(tickers)]
print(f"\nOnly {tickers} stocks:")
print(selected)
```

```
# =====
# 5. AGGREGATION (One value per column)
# =====
print("\n" + "=" * 60)
print("5. AGGREGATION")
print("=" * 60)
```

```
# Basic aggregation
print("Basic statistics:")
print(f"Mean price: ${df['Price'].mean():.2f}")
print(f"Median price: ${df['Price'].median():.2f}")
print(f"Total volume: {df['Volume'].sum()}")
print(f"Max price: ${df['Price'].max()}")
print(f"Min volume: {df['Volume'].min()}")
```

```
# describe() - all stats at once
print("\ndescribe() - Full statistical summary:")
print(df.describe())
```

```
# Multiple aggregations at once
print("\nMultiple aggregations on Price column:")
print(df['Price'].agg(['mean', 'median', 'std', 'min', 'max']))
```

```
# Groupby aggregation (finance classic - by ticker)
print("\nGroup by Ticker - Mean price per stock:")
print(df.groupby('Ticker')['Price'].mean())
```

```
# numeric_only example (what you mentioned)
print("\nAggregate only numeric columns:")
print(df.mean(numeric_only=True))
```

```
# to_string() - full data without dots
print("\nto_string() - Complete DataFrame:")
print(df.to_string())
```

```

# =====
# 6. DATA CLEANING (Your favorite part)
# =====
print("\n" + "=" * 60)
print("6. DATA CLEANING")
print("=" * 60)

# Create messy data for cleaning
messy_df = pd.DataFrame({
    'Ticker': ['aapl', 'AAPL', 'GOOGL', 'msft', 'MSFT', None],
    'Price': [150, None, 140, 380, 375, 100],
    'Volume': ['1,000', '1,200', None, '500', '600', '700'],
    'Sector': ['tech', 'TECH', 'Technology', 'tech', 'TECH', 'tech']
})

print("MESSY DATA:")
print(messy_df)

# 6a. Fix inconsistent strings (str.lower)
print("\n✓ After str.lower() on Ticker:")
messy_df['Ticker'] = messy_df['Ticker'].str.lower()
print(messy_df[['Ticker']])

# 6b. Remove commas from Volume and convert to int
print("\n✓ Remove commas from Volume:")
messy_df['Volume'] = messy_df['Volume'].str.replace(',', '').astype(float)
print(messy_df[['Volume']])

# 6c. Handle missing values (NaN)
print(f"\nMissing values before cleaning:\n{messy_df.isnull().sum()}")

# Fill missing Price with median
messy_df['Price'] = messy_df['Price'].fillna(messy_df['Price'].median())
print("\n✓ Filled missing Price with median")

# Drop rows where Ticker is missing
messy_df = messy_df.dropna(subset=['Ticker'])
print("\n✓ Dropped rows with missing Ticker")

# 6d. Replace values
messy_df['Sector'] = messy_df['Sector'].replace('Technology', 'tech')
messy_df['Sector'] = messy_df['Sector'].str.lower()
print("\n✓ Standardized Sector to lowercase")

```

```

print("\nCLEAN DATA:")
print(messy_df)

# =====
# 7. BONUS: REAL FINANCE WORKFLOW
# =====

print("\n" + "=" * 60)
print("7. REAL FINANCE WORKFLOW (All concepts combined)")
print("=" * 60)

# Create sample portfolio data
portfolio = pd.DataFrame({
    'Date': pd.date_range('2024-01-01', periods=10, freq='D'),
    'AAPL': np.random.normal(150, 5, 10),
    'GOOGL': np.random.normal(140, 4, 10),
    'MSFT': np.random.normal(380, 8, 10)
})
portfolio.set_index('Date', inplace=True)

print("Portfolio prices (first 5 days):")
print(portfolio.head())

# Filter: Days where AAPL > 150
high_aapl = portfolio[portfolio['AAPL'] > 150]
print(f"\nDays with AAPL > $150: {len(high_aapl)} days")

# Aggregation: Daily portfolio average
print(f"\nAverage price per stock:")
print(portfolio.mean())

# Data cleaning in real scenario
portfolio_clean = portfolio.dropna() # Remove any missing days
portfolio_clean = portfolio_clean[portfolio_clean['AAPL'] > 0] # Remove negative

# Filtering + Aggregation combo
portfolio['Total'] = portfolio.sum(axis=1)
high_days = portfolio[portfolio['Total'] > 680]
print(f"\nDays with total portfolio value > $680: {len(high_days)}")

print("\n" + "=" * 60)
print("✅ You've used EVERY concept from your pandas diary!")
print("=" * 60)
print("Concepts demonstrated:")

```

```
print("✓ DataFrames & Series | ✓ Dictionary to Series | ✓ loc & iloc")
print("✓ CSV/JSON import/export | ✓ Filtering (without if-else)")
print("✓ Aggregation (mean, median, sum, describe, groupby)")
print("✓ to_string() | ✓ numeric_only | ✓ Data cleaning")
print("✓ str.lower() | ✓ fillna() | ✓ dropna() | ✓ replace()")
```

Matplotlib:

Nothing fancy about this just a visualiser of anything u have data on by bar diagrams, pie charts, scatter plots, histograms, subplots and most useful when combined and used with pandas

Plot Customisation:

U can customize a lot of features mainly title, colors, grid lines, label names and many more features to make it more understanding top the viewers and can know the patterns in the data

Labels:

They are some sort of identification factor of what was being plotted in there and we can actually modify the colors ,that point of intersection and all

Grid Lines:

They are some lines not exactly related to what was being plot and just shows us lines aligning with the vertical and horizontal scales depending on what we want like if only horizontal or vertical or both

Bar Charts:

One type of data interpretation to show what was in the data and u can use features like labelling them ,coloring the lines,titles and many more

Pie Charts:

Same as bar plots nothing to say but we can use special functions like explode to split some part of the pie

Scatter Plots:

Nothing to say just an interpreter of data

Histograms:

Same here too

*one thing i wanna say is they are unique in their way but their purpose is same finding patterns or visualising the given data for understanding

Subplots:

We can use figure,axis function to split the canvas and add many graphs on that for showing multiple things on the same canvas so yeah a special function which is used to plotting two or many graphs at the same time

Pandas and Matplotlib:

Matplotlib on their own isn't much of a useful thing anyway because without data u r not plotting anything and data is created by filtering , aggregating,and lastly data cleaning by pandas and then data is used to visualise it

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# =====
# CREATE FINANCE DATA (Pandas first, as you said)
# =====
print("=" * 60)
print("STEP 1: PANDAS DOES THE WORK (cleaning, filtering, aggregation)")
print("=" * 60)

# Generate 100 days of stock data for 3 companies
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=100, freq='D')

data = {
    'Date': dates,
    'AAPL': 150 + np.random.normal(0, 2, 100).cumsum(),
    'GOOGL': 140 + np.random.normal(0, 1.5, 100).cumsum(),
    'MSFT': 380 + np.random.normal(0, 3, 100).cumsum(),
    'Volume': np.random.randint(1000, 10000, 100)
}

df = pd.DataFrame(data)
df.set_index('Date', inplace=True)

print("First 5 rows of cleaned data:")
print(df.head())

# Calculate daily returns
returns = df.pct_change().dropna()

print("\n" + "=" * 60)
print("STEP 2: MATPLOTLIB VISUALIZES EVERYTHING")
```

```

print("=" * 60)

# =====
# 1. BASIC PLOT WITH CUSTOMIZATION
# =====

plt.figure(figsize=(12, 6))

# Plot with custom colors
plt.plot(df.index, df['AAPL'], label='Apple', color='green', linewidth=2)
plt.plot(df.index, df['GOOGL'], label='Google', color='blue', linewidth=2, linestyle='--')
plt.plot(df.index, df['MSFT'], label='Microsoft', color='red', linewidth=1.5, marker='o',
markersize=2)

# CUSTOMIZATION: Title, labels, grid
plt.title('Stock Prices Over Time (2024)', fontsize=16, fontweight='bold')
plt.xlabel('Date', fontsize=12)
plt.ylabel('Price ($)', fontsize=12)
plt.grid(True, alpha=0.3, linestyle='--') # Grid lines you mentioned
plt.legend(loc='upper left')

plt.tight_layout()
plt.show()

# =====
# 2. BAR CHART (Final prices comparison)
# =====

final_prices = df.iloc[-1] # Last day's prices

plt.figure(figsize=(8, 6))
bars = plt.bar(final_prices.index, final_prices.values, color=['green', 'blue', 'red'])

# Customization
plt.title('Final Stock Prices (Day 100)', fontsize=14, fontweight='bold')
plt.xlabel('Company', fontsize=12)
plt.ylabel('Price ($)', fontsize=12)
plt.grid(True, axis='y', alpha=0.3) # Only horizontal grid lines

# Add value labels on bars
for bar, value in zip(bars, final_prices.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
             f'${value:.2f}', ha='center', fontweight='bold')

plt.tight_layout()
plt.show()

```



```

# =====
# 3. PIE CHART (Market share by final price - just for fun)
# =====
plt.figure(figsize=(8, 8))
explode = (0.05, 0, 0) # Explode Apple slice slightly

plt.pie(final_prices.values,
        labels=final_prices.index,
        explode=explode,
        autopct='%1.1f%%',
        colors=['green', 'blue', 'red'],
        shadow=True,
        startangle=90)

plt.title('Market Share by Price (Not real market cap, just demo)', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# 4. SCATTER PLOT (Volume vs Return)
# =====
# Calculate 7-day return
returns_7d = (df / df.shift(7) - 1) * 100
latest_volume = df['Volume'].iloc[-50:] # Last 50 days volume
latest_returns = returns_7d['AAPL'].iloc[-50:] # Last 50 days returns

plt.figure(figsize=(10, 6))
plt.scatter(latest_volume, latest_returns, alpha=0.6, color='purple', s=50)

plt.title('Trading Volume vs 7-Day Returns (AAPL)', fontsize=14, fontweight='bold')
plt.xlabel('Volume (shares)', fontsize=12)
plt.ylabel('7-Day Return (%)', fontsize=12)
plt.grid(True, alpha=0.3)

# Add trend line (simple)
z = np.polyfit(latest_volume, latest_returns, 1)
p = np.poly1d(z)
plt.plot(latest_volume, p(latest_volume), "r--", alpha=0.8, label='Trend')

plt.legend()
plt.tight_layout()
plt.show()

```

```

# =====
# 5. HISTOGRAM (Distribution of daily returns)
# =====
plt.figure(figsize=(10, 6))

# Histogram with customization
plt.hist(returns['AAPL'], bins=30, alpha=0.5, color='green', edgecolor='black', label='AAPL')
plt.hist(returns['GOOGL'], bins=30, alpha=0.5, color='blue', edgecolor='black', label='GOOGL')
plt.hist(returns['MSFT'], bins=30, alpha=0.5, color='red', edgecolor='black', label='MSFT')

plt.title('Distribution of Daily Returns', fontsize=14, fontweight='bold')
plt.xlabel('Daily Return (%)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(True, axis='y', alpha=0.3)
plt.legend()

# Add vertical line at zero
plt.axvline(x=0, color='black', linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

# =====
# 6. SUBPLOTS (Multiple graphs on same canvas)
# =====
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Stock Analysis Dashboard', fontsize=16, fontweight='bold')

# Subplot 1: Price trend
axes[0, 0].plot(df.index, df['AAPL'], color='green', linewidth=1.5)
axes[0, 0].set_title('AAPL Price Trend')
axes[0, 0].set_ylabel('Price ($)')
axes[0, 0].grid(True, alpha=0.3)

# Subplot 2: Returns distribution
axes[0, 1].hist(returns['AAPL'], bins=30, color='green', alpha=0.7, edgecolor='black')
axes[0, 1].axvline(x=0, color='red', linestyle='--')
axes[0, 1].set_title('AAPL Returns Distribution')
axes[0, 1].set_xlabel('Return (%)')

# Subplot 3: Volume over time
axes[1, 0].bar(df.index[-30:], df['Volume'].iloc[-30:], color='blue', alpha=0.7)
axes[1, 0].set_title('Trading Volume (Last 30 days)')
axes[1, 0].set_ylabel('Volume')

```

```

axes[1, 0].grid(True, axis='y', alpha=0.3)
plt.setp(axes[1, 0].xaxis.get_majorticklabels(), rotation=45)

# Subplot 4: Scatter (Volume vs Return)
axes[1, 1].scatter(df['Volume'].iloc[-50:], returns['AAPL'].iloc[-50:], alpha=0.5, color='purple')
axes[1, 1].set_title('Volume vs Return (AAPL)')
axes[1, 1].set_xlabel('Volume')
axes[1, 1].set_ylabel('Return (%)')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# =====
# 7. PANDAS + MATPLOTLIB (Direct plotting)
# =====
print("\n" + "=" * 60)
print("STEP 3: PANDAS DIRECT PLOTTING (Even easier)")
print("=" * 60)

# Pandas has built-in plotting!
df[['AAPL', 'GOOGL', 'MSFT']].plot(figsize=(12, 6), title='Stock Prices (Pandas plot method)')
plt.grid(True, alpha=0.3)
plt.ylabel('Price ($)')
plt.tight_layout()
plt.show()

# Rolling average with pandas + matplotlib
df['AAPL_MA20'] = df['AAPL'].rolling(window=20).mean()

plt.figure(figsize=(12, 6))
plt.plot(df.index, df['AAPL'], label='AAPL Actual', alpha=0.5, linewidth=1)
plt.plot(df.index, df['AAPL_MA20'], label='20-Day MA', color='red', linewidth=2)
plt.title('AAPL with 20-Day Moving Average', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Price ($)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("✅ You've used EVERY concept from your Matplotlib diary!")
print("=" * 60)

```

```
print("Concepts demonstrated:")
print("✓ Basic plots with customization | ✓ Labels (title, xlabel, ylabel)")
print("✓ Grid lines (with alpha, linestyle) | ✓ Bar charts with value labels")
print("✓ Pie charts with explode | ✓ Scatter plots with trend line")
print("✓ Histograms with distributions | ✓ Subplots (2x2 grid)")
print("✓ Pandas + Matplotlib integration | ✓ Rolling averages")
```

"Matplotlib on their own isn't useful – data is created by filtering, aggregating, cleaning by pandas"

This is exactly right. The workflow is:

Raw Data → Pandas (clean, filter, aggregate) → Matplotlib (visualize)